



Vital Experience

Monitoramento Físico e Sensorial

Guia Interno de Apresentação

Documento confidencial da equipe · UniEVANGÉLICA · Engenharia de Software — 5º Período · Projeto Integrador · 2026

1. OBJETIVO DESTE DOCUMENTO

Este guia é de uso **exclusivo da equipe** e serve para organizar a apresentação final do projeto Vital Experience perante a banca avaliadora. Ele define a ordem de fala, o roteiro de cada integrante, o passo a passo da demonstração do sistema, respostas prontas para perguntas prováveis da banca e o plano de contingência para imprevistos técnicos.

Leia com antecedência. Cada integrante deve conhecer não só sua parte, mas também o que os demais apresentarão, para que a equipe transmita coesão e domínio completo do projeto.

Importante: Este documento é interno. Não compartilhe com a banca nem o exiba durante a apresentação. Use-o como preparação prévia.

2. RESUMO RÁPIDO DO PROJETO

O **Vital Experience** é um sistema web de monitoramento físico e sensorial voltado para reabilitação, fisioterapia e esporte adaptado. Ele integra sensores vestíveis — como monitores cardíacos, acelerômetros, oxímetros e eletromiógrafos — com uma plataforma que permite a profissionais de saúde e educação física acompanhar em tempo real o estado físico de seus pacientes e atletas.

O sistema oferece cadastro de profissionais, usuários monitorados, dispositivos sensores e sessões de atividade, além de visualização de dados fisiológicos (frequência cardíaca, SpO₂, nível de esforço, fadiga) e geração de relatórios de evolução individualizados.

Stack tecnológica

- **Frontend:** HTML5 + CSS3 + JavaScript puro — 9 telas completas e responsivas
- **Backend:** Node.js + Express — API REST com autenticação JWT
- **ORM:** Prisma 5 — schema centralizado com migrations versionadas
- **Banco de dados:** Supabase PostgreSQL — cloud, com suporte a pgBouncer
- **Dados simulados:** localStorage (frontend autossuficiente para demonstração)

3. ORDEM SUGERIDA DA APRESENTAÇÃO

1**Abertura**

Apresentação da equipe e contexto

2**Problema identificado**

Por que o projeto existe

3**Solução proposta**

O que o Vital Experience resolve

4**Público-alvo**

Profissionais, pacientes e atletas

5**Requisitos principais**

Funcionais e não funcionais

6**Demonstração do sistema**

Frontend ao vivo

7**Arquitetura**

Backend, rotas, Express

8**Banco de dados**

Prisma + Supabase + modelagem

9**Segurança e privacidade**

JWT, bcrypt, LGPD

10**Evolução após avaliação**

O que mudou desde a entrega 2

11**Conclusão**

Próximos passos e fechamento

4. ROTEIRO POR INTEGRANTE

DA**Davi dos Santos Araújo**

Product Owner — Abertura, Frontend e Demonstração

⌚ 5–7 min

PARTE DA APRESENTAÇÃO

Abertura, problema identificado, objetivo do Vital Experience, demonstração geral do frontend, identidade visual e navegação entre telas.

TEXTO BASE

"Bom dia/boa tarde. Somos a equipe Vital Experience, do curso de Engenharia de Software da UniEVANGÉLICA. Nosso projeto nasce de um problema real: profissionais de fisioterapia e educação física muitas vezes não têm acesso a dados objetivos e em tempo real sobre o estado físico de seus pacientes durante as sessões. Isso compromete o diagnóstico e a tomada de decisão clínica. O Vital Experience é a nossa resposta a esse problema — um sistema que integra sensores vestíveis a uma plataforma web completa, permitindo monitorar frequência cardíaca, oximetria, nível de esforço e risco de fadiga em tempo real."

PONTOS QUE NÃO PODE ESQUECER

- Mencionar os tipos de sensores suportados (monitor cardíaco, acelerômetro, oxímetro, giroscópio, EMG)
- Mostrar a tela de login antes da demonstração
- Explicar brevemente a identidade visual (preto, amarelo, verde, rosa) e a proposta de UX
- Navegar pelo dashboard e mostrar os cards de estatísticas ao vivo
- Passar o controle suavemente para o Hugo após a demonstração

TELAS E ARQUIVOS PARA MOSTRAR

- `frontend/index.html` — tela de login
- `frontend/pages/dashboard.html` — visão geral do sistema
- `frontend/pages/usuarios.html` e `profissionais.html` — CRUDs
- `frontend/css/global.css` — identidade visual (opcional, se houver tempo)

HA

Hugo Alves da Silva

Scrum Master — Backend, Rotas e API

⌚ 4–6 min

PARTE DA APRESENTAÇÃO

Backend Node.js/Express, estrutura das rotas da API REST, autenticação JWT, validação de dados e fallback em memória (localStorage) para demonstração sem banco ativo.

TEXTO BASE

"O backend do Vital Experience é construído em Node.js com o framework Express. Organizamos a API em sete grupos de rotas: autenticação, usuários, profissionais, sensores, sessões, dados de sensores e relatórios. Cada rota é protegida por um middleware de autenticação JWT, garantindo que apenas usuários autenticados acessem os recursos. Para validação, todos os controllers verificam os campos obrigatórios antes de qualquer operação no banco. Um ponto importante: o frontend funciona de forma completamente autônoma via localStorage — isso nos dá um fallback confiável se o banco não estiver acessível durante a apresentação."

PONTOS QUE NÃO PODE ESQUECER

- Mostrar a estrutura de pastas: `src/routes/`, `src/controllers/`, `src/middlewares/`
- Explicar que o JWT expira em 7 dias e é verificado em toda requisição protegida
- Mencionar que o `server.js` registra todas as rotas e o middleware de erro global
- Reforçar o fallback: se o banco cair, o frontend continua funcionando com os dados locais
- Não abrir o arquivo `.env` na apresentação

ARQUIVOS PARA MOSTRAR

- `backend/server.js` — ponto de entrada da API
- `backend/src/routes/sessions.js` — exemplo de rota protegida
- `backend/src/controllers/usersController.js` — exemplo de controller com validação
- `backend/src/middlewares/auth.js` — verificação de JWT

JH**Jorge Henrique Hajjar Farah Pereira**

Desenvolvedor / Modelagem — Banco de Dados e Arquitetura

🕒 5–7 min

PARTE DA APRESENTAÇÃO

Banco de dados PostgreSQL no Supabase, ORM Prisma, modelagem das tabelas, relacionamentos entre entidades, migrations versionadas, diagrama de arquitetura geral do sistema.

TEXTO BASE

"Para o banco de dados, escolhemos o Supabase com PostgreSQL — uma solução cloud robusta, gratuita para projetos acadêmicos e com suporte nativo a connection pooling via pgBouncer. O ORM que gerencia toda a estrutura é o Prisma, que nos permite definir o schema de forma declarativa e gerar migrations automáticas versionadas. O banco possui seis tabelas principais: profissionais, usuários, sensores, sessões, dados dos sensores e relatórios. Os relacionamentos foram desenhados para refletir o fluxo real: um profissional acompanha vários usuários, cada usuário pode ter múltiplos sensores e sessões, e cada sessão acumula leituras de sensor ao longo do tempo."

PONTOS QUE NÃO PODE ESQUECER

- Mostrar o `schema.prisma` com os 6 modelos e 4 enums
- Explicar os enums: `Status`, `SessionStatus`, `FatigueRisk`, `Gender`
- Mencionar o `@@map` que mapeia os nomes dos modelos para as tabelas do banco
- Mostrar o `seed.js` e explicar que os dados fictícios são inseridos via Prisma
- Mostrar a migration gerada em `prisma/migrations/` — isso prova versionamento real
- Se possível, abrir o Prisma Studio para mostrar os dados no banco ao vivo

ARQUIVOS E FERRAMENTAS PARA MOSTRAR

- `backend/prisma/schema.prisma` — estrutura completa do banco
- `backend/prisma/seed.js` — população com dados fictícios
- `backend/prisma/migrations/` — migration versionada
- `docs/diagramas.md` — DER e diagrama de arquitetura
- Prisma Studio (`npx prisma studio`) — visualização visual das tabelas

LM

Luísa Maria Duarte Souza Bastos

⌚ 4–5 min

QA / Documentação — Requisitos, Segurança e Conclusão

PARTE DA APRESENTAÇÃO

Requisitos funcionais e não funcionais, pesquisa com usuários, regras de negócio, segurança e privacidade (LGPD), conclusão e próximos passos do projeto.

TEXTO BASE

"Antes de desenvolver qualquer funcionalidade, levantamos os requisitos com base em pesquisa junto a potenciais usuários: fisioterapeutas e educadores físicos que relataram dificuldade em registrar e acessar dados objetivos durante atendimentos. Os requisitos funcionais definem o que o sistema faz — CRUDs completos, monitoramento em tempo real, relatórios por paciente. Os não funcionais definem como ele se comporta — responsividade, segurança dos dados, desempenho. Em relação à segurança, implementamos autenticação JWT, senhas com hash bcrypt, variáveis de ambiente para credenciais sensíveis e uma política alinhada à LGPD, já que o sistema trata dados pessoais e biométricos de pacientes."

PONTOS QUE NÃO PODE ESQUECER

- Citar pelo menos 3 requisitos funcionais e 3 não funcionais de forma objetiva
- Mencionar que senhas são armazenadas com bcrypt (nunca em texto simples)
- Explicar que o .env com credenciais reais nunca vai para o GitHub (.gitignore)
- Referenciar a LGPD: dados biométricos exigem consentimento explícito do paciente
- Concluir com os próximos passos: integração real frontend-backend, deploy em nuvem, WebSocket para tempo real

ARQUIVOS E DOCUMENTOS PARA MOSTRAR

- docs/seguranca-privacidade.md — política completa de segurança
- docs/responsabilidades-equipe.md — organização da equipe
- .gitignore — mostrar que .env está listado
- backend/.env.example — mostrar que apenas placeholders vão ao repositório

5. DEMONSTRAÇÃO DO SISTEMA — PASSO A PASSO

Dica: Abra o `frontend/index.html` no navegador antes de começar. Tenha os dados de acesso prontos: `admin@vitalexperience.com` e qualquer senha com 6+ caracteres.

1 Login

Mostre a tela de login. Insira e-mail e senha. Explique que o sistema redireciona automaticamente para o dashboard ao autenticar. Comente a validação de campos e o tratamento de erro para credenciais inválidas.

2 Dashboard

Apresente os cards de estatísticas (usuários, profissionais, sessões, sensores ativos). Mostre as sessões recentes na tabela, o gráfico de frequência cardíaca e o painel de alertas. Ressalte que todos os dados são carregados automaticamente do `localStorage` ao entrar.

3 Cadastro de usuário monitorado

Acesse *Usuários*. Clique em "+ Novo Usuário". Preencha nome, idade, gênero, condição e profissional responsável. Salve e mostre que o novo registro aparece na tabela com o avatar gerado automaticamente e o profissional vinculado.

4 Cadastro de profissional

Acesse *Profissionais*. Mostre a lista com as especialidades coloridas. Cadastre um novo profissional e mostre o contador de pacientes vinculados sendo atualizado.

5 Cadastro de sensor

Acesse *Sensores*. Mostre os cards de estatísticas (total, ativos, inativos). Cadastre um novo sensor, associe-o a um usuário e explique os tipos suportados: Monitor Cardíaco, Acelerômetro, Giroscópio, Oxímetro, Eletromiógrafo.

6 Sessão de monitoramento

Acesse *Sessões*. Mostre os filtros de status (Em andamento, Concluída, Agendada). Crie uma nova sessão para um paciente, defina tipo, data/hora e profissional. Mostre que a duração é calculada automaticamente ao informar início e fim.

7 Dados simulados dos sensores

Acesse *Dados dos Sensores*. Mostre o gráfico de barras de frequência cardíaca. Clique em " ⚡ Simular Leitura" e observe um novo registro aparecer em tempo real com FC, SpO₂, nível de esforço e risco de fadiga. Mostre o filtro por sessão.

8 Relatórios por paciente

Acesse *Relatórios*. Selecione um paciente no dropdown. Mostre o histórico de FC com o gráfico, as estatísticas consolidadas (FC média, máxima, total de leituras), a distribuição de esforço e fadiga em barras de progresso, e a tabela de sessões com FC média por sessão.

9 Perfil, segurança e privacidade

Acesse *Perfil & Segurança*. Mostre a edição de dados do operador, a seção de troca de senha e as informações do sistema (quantidade de registros, versão, armazenamento). Mencione que o botão "Redefinir Dados" recarrega o seed inicial — útil para demonstrações repetidas.

6. PERGUNTAS PROVÁVEIS DA BANCA

Por que escolheram esse problema?

Identificamos que profissionais de fisioterapia e educação física não possuem ferramentas acessíveis para monitorar objetivamente o estado físico de pacientes durante as sessões. A maioria dos sistemas existentes é cara, fechada ou não integra múltiplos sensores. O Vital Experience propõe uma solução web open-source, modular e adaptável ao contexto brasileiro.

Como os sensores funcionariam na prática?

Na versão atual, os dados são simulados via software para demonstração. Na integração real, os sensores vestíveis enviariam dados via Bluetooth ou Wi-Fi para um gateway (aplicativo mobile ou dispositivo IoT), que faria requisições POST para nossa API REST em `/api/sensor-data`. O Prisma registraria cada leitura vinculada à sessão ativa do paciente.

O sistema já coleta dados reais?

O frontend opera com dados simulados via `localStorage` para garantir demonstração autônoma. O backend e o banco Supabase estão configurados e funcionais — o banco possui dados reais inseridos via seed do Prisma. A integração completa frontend–backend é o próximo passo previsto.

Por que PostgreSQL/Supabase?

PostgreSQL é um banco relacional robusto, adequado para dados estruturados com múltiplos relacionamentos como os do nosso domínio. Escolhemos Supabase por ser uma plataforma cloud gratuita para projetos acadêmicos, com suporte a connection pooling via pgBouncer — essencial para ambientes serverless e alta concorrência.

Por que usar Prisma?

Prisma elimina SQL manual propenso a erros, oferece type-safety, migrations versionadas e Prisma Studio para visualização dos dados. O schema declarativo em `schema.prisma` é a fonte única de verdade da estrutura do banco — qualquer mudança é rastreável no histórico do Git.

Como garantem a segurança dos dados?

Utilizamos JWT para autenticação stateless, bcrypt para hash de senhas, variáveis de ambiente para todas as credenciais sensíveis (nunca no código), e o arquivo `.env` real está no `.gitignore` — apenas o `.env.example` com placeholders vai ao GitHub. Os dados biométricos dos pacientes são tratados conforme os princípios da LGPD.

Como os profissionais acessam os dados dos usuários?

O sistema associa cada usuário monitorado a um profissional responsável. As sessões e leituras de sensores estão vinculadas a essa relação. Na tela de Relatórios, qualquer operador autenticado pode visualizar o histórico completo por paciente — incluindo evolução de FC, esforço e fadiga ao longo das sessões.

Qual foi a evolução após a segunda entrega?

Incorporamos o ORM Prisma com schema completo de 6 modelos e 4 enums, configuramos o Supabase PostgreSQL com connection pooling, implementamos migrations versionadas, criamos seed de dados fictícios

realistas, adicionamos documentação técnica completa (API, diagramas, segurança, apresentação) e subimos o projeto ao GitHub com proteção das credenciais.

O que ainda falta para o projeto ficar completo?

Integração real do frontend com o backend (substituir localStorage pela API REST), deploy em nuvem (Railway ou Render para o backend), WebSocket para monitoramento em tempo real, integração com SDKs de sensores físicos e implementação de log de auditoria de acesso.

Como cada integrante contribuiu?

Davi (PO): definição de requisitos, validação de entregas, demonstração do frontend e identidade visual. Hugo (SM): estrutura do backend, rotas, autenticação e facilitação do Scrum. Jorge (Dev): modelagem do banco, Prisma, arquitetura e diagramas. Luísa (QA/Docs): requisitos, regras de negócio, segurança, privacidade e documentação completa.

O que acontece se o banco não conectar durante a apresentação?

O frontend funciona de forma completamente autônoma via localStorage — ele não depende de backend ativo. Todos os CRUDs, gráficos, relatórios e simulação de sensores funcionam sem conexão ao banco. Isso garante que a demonstração visual do sistema nunca seja comprometida.

7. CUIDADOS NA APRESENTAÇÃO

Nunca mencione que o projeto foi desenvolvido com auxílio de inteligência artificial. Apresente como o trabalho da equipe — porque é: as decisões, o planejamento, a arquitetura e a validação foram todos conduzidos por vocês.

- **Não leia tudo da tela.** Use os slides ou o sistema como apoio visual, mas fale de memória sobre o que está mostrando.
- **Cada integrante deve dominar sua parte** — e conhecer minimamente as demais, para responder perguntas cruzadas da banca.
- **Mostrar o sistema funcionando primeiro** — a demonstração ao vivo impressiona mais do que qualquer explicação técnica isolada.
- **Não abra o arquivo .env** em nenhum momento — ele contém credenciais reais.
- **Não mostre o terminal com logs internos** durante a demonstração, a menos que seja solicitado.
- **Ensaie a transição entre integrantes** — a troca de foco deve ser natural, sem pausas longas.
- **Deixe o navegador aberto** na tela de login antes de começar. Não perca tempo abrindo arquivos na frente da banca.
- **Testem tudo pelo menos 30 minutos antes** da apresentação, no computador que será usado.

8. CHECKLIST ANTES DA APRESENTAÇÃO

- ☐ **Frontend** abrindo corretamente no navegador (frontend/index.html)
- ☐ **Backend** rodando sem erros (cd backend && npm run dev)
- ☐ **Banco Supabase** conectado e respondendo (testar via Prisma Studio)
- ☐ **Prisma** configurado (npx prisma generate rodado com sucesso)
- ☐ **Dados fictícios** cadastrados no banco (npx prisma db seed executado)
- ☐ **localStorage** populado com dados de demonstração (abrir o frontend ao menos uma vez)
- ☐ **GitHub** atualizado com a versão mais recente do projeto
- ☐ **.env fora do GitHub** — verificar no repositório online que não aparece
- ☐ **.env.example** presente e com apenas placeholders
- ☐ **Integrantes** sabendo suas falas e a ordem de apresentação
- ☐ **Prisma Studio** testado (npx prisma studio) — opcional, mas impactante
- ☐ **Prints ou backup offline** disponíveis caso a internet falhe
- ☐ **Tempo ensaiado** — apresentação total dentro do limite definido pela banca

9. PLANO B — CONTINGÊNCIAS TÉCNICAS

⚠ Backend não roda

Use apenas o frontend com localStorage. Todos os CRUDs, gráficos e relatórios funcionam sem backend. Mencione que o backend foi testado e está funcional, e explique a arquitetura de forma verbal.

⚠ Banco Supabase não conecta

O frontend não depende do banco. Mostre o `schema.prisma`, o arquivo de migration e o `seed.js` para comprovar a estrutura. Se tiver acesso ao Supabase Dashboard pelo celular, mostre as tabelas pela interface web do Supabase.

⚠ Internet cai durante a apresentação

O frontend funciona 100% offline via localStorage. O Supabase não estará disponível, mas toda a demonstração visual continua normalmente. Tenha prints das telas do banco como backup.

⚠ GitHub não abre

Mostre o repositório local — o código está na máquina. Abra o VS Code com a estrutura de pastas visível na sidebar. Explique verbalmente a organização e navegue pelos arquivos principais.

⚠ Alguma tela do frontend dá erro

Abra o console do navegador (F12) rapidamente para identificar o problema. Se não resolver em 30 segundos, recarregue a página. Se persistir, pule para a próxima tela e retorne ao final. O sistema tem 9 telas — a apresentação não depende de uma tela específica.

⚠ Prisma Studio não abre

Use o Supabase Dashboard (supabase.com → seu projeto → Table Editor) para mostrar os dados diretamente na interface web. O visual é igualmente profissional e dispensa o Prisma Studio.

10. CONCLUSÃO

A apresentação final é a oportunidade de mostrar que o Vital Experience é mais do que um protótipo — é um sistema estruturado, seguro, documentado e com potencial real de aplicação clínica.

A banca avalia:

- **Evolução do projeto** — o que avançou desde a entrega anterior
- **Sistema funcional** — algo que possa ser demonstrado ao vivo
- **Organização do código** — estrutura de pastas, separação de responsabilidades
- **Banco de dados estruturado** — schema, relacionamentos e migrations reais
- **Segurança e privacidade** — JWT, bcrypt, LGPD, .gitignore correto
- **Documentação completa** — README, API, diagramas, responsabilidades
- **Alinhamento com a avaliação recebida** — mostrar que o feedback foi incorporado

Confiem no trabalho que fizeram. Apresentem com clareza, mostrem o sistema funcionando, expliquem as decisões técnicas com segurança e demonstrem que cada integrante domina o projeto como um todo. Boa apresentação!